

Labo 6

Auteurs :

- Patrick Furrer
- Simon Guggisberg
- Jonas Troeltsch

Rapport

Explication de l'implémentation

Interface utilisateur

Pour l'implémentation de l'interface utilisateur, nous avons choisi de ne pas utiliser Jetpack Compose et d'utiliser la méthode traditionnelle avec XML et Kotlin Ceci a été décidé en raison de la charge de travail d'autres matières et du manque de temps pour explorer Jetpack Compose.

Nous avons accepté les dates sous forme de `String`, plutôt que de `DatePicker`, afin de simplifier l'implémentation. Le format de date dans `EditContactFragment` est sous la forme : `yyyy-MM-dd`. De plus, le fragment `EditContactFragment` est également responsable de la création de nouveaux contacts, la distinction se fait via `selectedContact` dans `ContactsViewModel`. Si nous sommes en cas de modification, alors trois boutons sont affichés : `Save`, `Cancel` et `Delete`. Dans le cas contraire, uniquement deux : `Create` et `Cancel`.

Contact

Pour rendre les contacts compatibles avec un sync vers la database distante, nous avons ajouté 2 attributs supplémentaires : `status` et `serverId`.

- `status` est un enum qui peut prendre les valeurs `NEW`, `UPDATED`, `DELETED` ou `OK`.
- `serverId` est l'id du contact sur le serveur.

Afin de pouvoir transformer les JSON reçus en objet `Contact`, nous avons créé une classe `ContactDTO` qui contient les mêmes attributs que la classe `Contact` mais avec des types de données différents. Nous avons ensuite créé une méthode `toContact()` qui permet de transformer un `ContactDTO` en `Contact`.

Remote Sync

La class `RemoteSyncManager` permet de synchroniser les contacts locaux avec le serveur. Elle contient les méthodes suivantes pour effectuer la synchronisation :

- `enroll()`
- `getToken()`
- `getAllContacts()`
- `getContact(id: Long)`
- `saveOrRegisterContact(contact: Contact)`
- `editContact(contact: Contact)`

- `registerContact(contact: Contact)`
- `deleteContact(contact: Contact)`

Le `ContactRepository` change le status des contacts puis appelle les méthodes de `RemoteSyncManager` pour effectuer la synchronisation et les sauvegarde en local.

Pour déterminer si une requête est un succès nous vérifions le code de réponse HTTP. Cela nous permet de vérifier si la requête est valide du côté du serveur. Dans ce cas là, le status du contact passe en "Ok".

`getToken()` retourne le token obtenue dans le liveData. À noter qu'avec notre implémentation actuelle, nous ne vérifions pas s'il est nul, vu que la `MainActivity` s'assure qu'un token soit mis en place.

Si l'on souhaite augmenter la robustesse, le `RemoteSyncManager` pourrait `enroll()` en cas de token `null`. Puisque la `MainActivity` observe la liveData, il devrait être mis à jour dans le `SharedPreferences`. Nous avons décider de garder la version actuelle pour garder le concept de eager loading.

Une implémentation comme celle-ci serait plus robuste :

```
private fun getToken() : String {
    var token = uuid.getData().value
    if (token == null) {
        token = enroll()
    }
    return token
}
```

A noter que dans `enroll()` le `remoteSyncManager` poste la nouvelle valeur dans le liveData. Cela signifie que notre implémentation actuelle ne permet **pas** d'avoir plusieurs UUID différentes. Transformer notre UUID livedata en tableau serait une approche pour permettre ceci, mais cela n'est pas nécessaire dans le cadre de ce projet.

SharedPreferences pour token

Nous avons utilisé les `SharedPreferences` pour stocker le token de l'utilisateur. Pour faciliter les interactions avec l'UUID nous avons décider de le mettre dans un livedata dans une classe dédiée, Ainsi si nous souhaitons l'utiliser dans d'autre composant, il est facile d'y accéder.

Pour garantir que l'UUID soit partagé correctement, nous avons décidé d'utiliser un compagnon object. Ainsi tout les instances accèdent à la même valeur.

L'activité principale s'occupe de charger l'UUID du cache (`SharedPreferences`). S'il n'existe pas, elle fait un appel à `enroll()` (défini dans `RemoteSyncManager`) pour obtenir un nouveau token. De plus elle observe la livedata pour mettre à jour le cache en cas de changement de valeur du token, typiquement si on décide de supprimer tout les données et partir à neuf (quand on appuie sur le bouton d'enroll).

ContactAdapter

Nous avons modifié le `ContactAdapter`, plus précisément le `ViewHolder` pour que les objets dans la liste aient une couleur différente en fonction de leur status par rapport au serveur distant.

- Bleu : Les objets créés mais pas encore présents sur le serveur.
- Orange : Ceux édités en local mais pas sur le serveur.
- Rouge : Ceux Supprimés en local (ils disparaissent uniquement lorsque la synchronisation est un succès)
- Vert : Ceux dont le statut est synchrone.

Implémentation des appels API 4.3

Pour l'implémentation de ces méthodes nous avons choisis `Java-net-url` pour sa simplicité d'utilisation.

Pour ce qui est de la synchronisation entre le serveur et la DB local, une première modification est faite en local avec un statut `Updated New` et `Deleted`, respectivement. Puis, à la réception d'une réponse positive du serveur, le statut local passe à `Ok`.

À noter que pour la suppression, le contact passe d'abord par `Deleted`, avant de, si le serveur a répondu `Ok`, de supprimer également localement.